Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 1.3

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 14/09/2026

Definición de funciones

Una función es un bloque de código diseñado para realizar una tarea específica. Una vez declarada, puede ser llamada dentro de nuestro programa todas las veces que se necesite realizar esta tarea nuevamente. Dentro de una función se puede escribir todo tipo de código, el cual, en algunas ocasiones puede ser utilizado para devolver algún resultado o simplemente para ejecutar una acción en específico. Las funciones nos permiten desarrollar programas más simples, fáciles de entender, de probar y de corregir, en caso de ser necesario (Matthes, 2019).

Una forma simple de escribir una función en Python puede ser de la siguiente manera (Tabla 1):

```
def nombre_de_la_funcion():
 pass
```

En el ejemplo anterior, la primera línea muestra la utilización de la palabra reservada def, la cual le indica a Python que se comenzará a describir una función; a continuación, se coloca el nombre de dicha función seguida de unos paréntesis cerrados y el símbolo de dos puntos. La segunda línea en el ejemplo también es una palabra reservada que significa saltar, y que en este caso utilizamos para definir que en esa sección se escribirá el código posteriormente. La sección donde escribimos el código interno de la función se conoce como el cuerpo de esta.

```
def  saludar():
  print("Buenos días queridos estudiantes")
```

En este caso, el nombre de la función es saludar y el cuerpo está compuesto por la instrucción de imprimir el texto indicado. Una función que solamente se ha declarado no tiene ninguna utilidad hasta que es llamada o invocada, para este caso la sintaxis correcta sería la siguiente

```
saludar()
```

```
Buenos días queridos estudiantes
```

Otro ejemplo simple que ilustra todo lo anterior puede ser

```
def despedir():
  print("Hasta luego, nos vemos pronto")
```

```
despedir()
```

```
Hasta luego, nos vemos pronto
```

Una función puede recibir diversos valores de entrada. Cuando las variables que representan esos valores se utilizan en la declaración de la función se les denominan “parámetros”; entonces es correcto expresar que en el siguiente ejemplo la función saludar() recibe el parámetro de entrada “nombre”

```
def  saludar(nombre):
  print(f"Buenos dias {nombre}, ¿Cómo estas hoy?")
```

El valor que se le pasa a la función durante la llamada a la misma se le denomina argumento. De esa forma se puede expresar que en el siguiente ejemplo la función saludar() es invocada con el argumento “María”

```
def  saludar(nombre):
    print(f"Buenos dias {nombre}, ¿Cómo estas hoy?")

saludar("María")
```

```
Buenos dias María, ¿Cómo estas hoy?
```

Múltiples parámetros y argumentos

Las funciones pueden ser declaradas con múltiples parámetros de entrada, por lo cual al momento de llamarlas es necesario utilizar de igual forma múltiples argumentos. Esta operación se puede realizar de distintas formas, a continuación, veremos algunas de ellas (Matthes, 2019):

Argumentos posicionales

Cuando se invoca a una función, Python hace coincidir los argumentos recibidos con los parámetros declarados. La forma más sencilla de lograr esto es manteniendo el orden en que aparecen los elementos

```
def saludar(nombre, dia_de_la_semana):
  print(f"Buenos dias {nombre}, feliz {dia_semana}")
```

Donde las llamadas a la función pudieran ser las siguientes

```
def saludar(nombre, dia_semana):
  print(f"Buenos dias {nombre}, feliz {dia_semana}")

saludar("Arturo", "sábado")
saludar("Juan", "domingo")
```

```
Buenos dias Arturo, feliz sábado
Buenos dias Juan, feliz domingo
```

Argumentos de palabras clave

En los ejemplos vistos anteriormente si invertimos el orden de los argumentos en las llamadas de las funciones, nuestro programa no va a generar errores, pero los resultados no serán adecuados

```
saludar("sábado","Arturo")
```

```
Buenos dias sábado, feliz Arturo
```

Para solucionar esta problemática se utilizan los argumentos de palabras claves, los cuales consisten en un par nombre-valor que se le pasa a la función a la hora de invocarla. De esta manera se realiza la asociación directa, sin dar pie a confusiones. Una nueva forma invocar la función pudiera ser

```
saludar(nombre="Juan", dia_semana="sábado")
saludar( dia_semana="sábado", nombre="Juan")
```

```
Buenos dias Juan, feliz sábado
Buenos dias Juan, feliz sábado
```

Donde ambas darían iguales resultados, sin importar el orden de los argumentos.

Valores por defecto

Pudiera darse el caso de que al momento de invocar la función omitiéramos alguno de sus argumentos, en esa situación nuestro programa va a generar un error. En situaciones como estas o donde también se puede considerar que el argumento no es obligatorio, una buena práctica es declarar valores por defecto en los parámetros de la función

```
def saludar(nombre= "estudiante", dia_semana="semana"):
  print(f"Buenos dias {nombre}, feliz {dia_semana}")

saludar()
saludar("José")
```

```
Buenos dias estudiante, feliz semana
Buenos dias José, feliz semana
```

Según Matthes (2019), es importante señalar que, en caso de tener una función con múltiples parámetros, donde no todos tienen valores por defecto, la forma correcta es colocar primero los parámetros sin valores y a continuación el resto, ya que de esa forma le permitimos a Python interpretar los argumentos posicionales de forma correcta.

Uso de funciones

Valores de retorno

Las funciones tienen varios usos. Anteriormente vimos su aplicación para mostrar elementos de forma directa, pero también es muy común su utilización en la obtención de resultados que se manejaran posteriormente en otras partes del programa. A estos elementos se les conoce como valores de retorno. Una función con valores de retorno se puede escribir de la siguiente forma

```
def sumar(x,y):
  return x + y
```

El valor que se desea retornar va precedido por la palabra reservada “return”, la cual puede estar asociada a una variable o directamente a una operación, como el caso que se ilustró en el ejemplo anterior.

Cuando se llama a una función con valores de retorno estas deben estar asignadas a alguna variable donde se almacenará el contenido retornado

```
resultado = sumar(4, 8)
print(resultado)
```

```
12
```

Argumentos opcionales

Una aplicación específica puede requerir que no todos los argumentos de entrada estén declarados, en este caso es adecuado utilizar los argumentos opcionales. En el siguiente ejemplo se ilustra un posible caso de uso donde el segundo apellido es un argumento opcional

```
def datos_cliente(numero_cliente, nombre, primer_apellido, segundo_apellido = ''):
  print(f"El cliente {numero_cliente} se llama: {nombre} {primer_apellido} {segundo_apellido}")
```

Al momento de llamar la función podemos incluirlo o no, sin generar ningún error

```
datos_cliente(45, "Roberto", "López", "Valdéz")
datos_cliente(45, "Roberto", "López")
```

```
El cliente 45 se llama: Roberto López Valdéz
El cliente 45 se llama: Roberto López
```

Retornar un diccionario

Una función puede retornar cualquier tipo de valor, incluyendo estructuras de datos más complejas como los diccionarios. Supongamos que necesitamos obtener como resultado un diccionario con los datos referentes al cliente del ejemplo anterio

```
def datos_cliente (numero_cliente, nombre, primer_apellido):
    cliente = {'numero_cliente': numero_cliente,
               'nombre': nombre,
```

```
                'apellido': primer_apellido}
    return cliente

usuario = datos_cliente(45, "Roberto", "López")
print(usuario)
```

```
{'numero_cliente': 45, 'nombre': 'Roberto', 'apellido': 'López'}
```

De esta forma podemos convertir simple información textual en una estructura más cómoda de utilizar y procesar en otras secciones de nuestro programa.

Funciones y listas

Otro uso muy útil que tienen las funciones radica en pasar una lista como argumento. Una lista puede estar compuesta por números, textos o estructuras de datos más complejas, por lo que es de suma importancia tener claridad sobre cuáles son los valores con los que estamos trabajando y su respectivo uso

```
def saluda_equipo(nombres):
  for nombre in nombres:
    mensaje = f"Buenos días, {nombre.title()}."
    print(mensaje)

miembros_equipo = [ "Marcos", "David", "Juana", "Laura"]
saluda_equipo(miembros_equipo)
```

```
Buenos días, Marcos.
Buenos días, David.
Buenos días, Juana.
Buenos días, Laura.
```

Además de operar con el contenido de una lista, también podemos modificarlas y devolverlas con una estructura similar

```
def modificar_equipo(nombres, nuevo_miembro):
  equipo = nombres
  equipo.append(nuevo_miembro)

miembros_equipo = ["Marcos", "David", "Juana", "Laura"]
modificar_equipo(miembros_equipo, "Erick")
print(miembros_equipo)
```

```
['Marcos', 'David', 'Juana', 'Laura', 'Erick']
```

Cuando operamos con listas dentro de una función, estas pueden estar propensas a sufrir modificaciones. Este comportamiento en algunas aplicaciones puede no ser el más adecuado, por lo que Python proporciona una forma de evitar esta situación mediante el envío de una copia de la lista original

```
def modificar_equipo(nombres, nuevo_miembro):
  equipo = nombres
  equipo.append(nuevo_miembro)
  return equipo

miembros_equipo = ["Marcos", "David", "Juana", "Laura"]
nuevo_equipo = modificar_equipo(miembros_equipo[:], "Erick")
print(miembros_equipo)
print(nuevo_equipo)
```

```
['Marcos', 'David', 'Juana', 'Laura']
['Marcos', 'David', 'Juana', 'Laura', 'Erick']
```

Utilizar un número arbitrario de argumentos

Según Python Software Foundation (s. f.), en desarrollos más complejos puede ser que no tengamos control sobre todos los argumentos que acepta una función, para solucionar esta problemática Python nos permite declarar funciones con un número arbitrario de parámetros

```
def hacer_hamburguesa(*agregados):
  print(agregados)

hacer_hamburguesa("queso", "tocino", "jamón", "tomate")
```

```
('queso', 'tocino', 'jamón', 'tomate')
```

La utilización del carácter "*", genera una tupla vacía donde se almacenan todos los valores que le entran a la función durante su invocación.

Otra forma muy útil es la posibilidad de mezclar argumentos posicionales y argumentos arbitrarios, por ejemplo

```
def hacer_hamburguesa(tipo, *agregados):
  print(f"Hacer hamburguesa {tipo} con los siguientes agregados:")
  for agregado in agregados:
    print(f"-{agregado}")

hacer_hamburguesa("sencilla", "queso", "tocino", "jamón", "tomate")
```

```
Hacer hamburguesa sencilla con los siguientes agregados:
-queso
-tocino
-jamón
-tomate
```

Funciones Lambda

Python nos permite generar pequeñas funciones anónimas que pueden solucionar un problema particular sin tener que escribir toda una declaración formal. Este tipo de funciones se conocen como funciones lambda y son una de las características más poderosas de este lenguaje (Python Software Foundation, s. f.). La sintaxis que se utiliza es la siguiente:

lambda argumentos: expresión

En este caso, la sentencia comienza con la palabra reservada "lambda", seguida por los argumentos y la expresión que definirá el comportamiento de la función. A continuación, se muestra un ejemplo donde se utiliza una de estas funciones para realizar un sencillo cálculo

```
incrementar_5_unidades = lambda x: x + 5
incrementar_5_unidades(16)
```

```
21
```

Guardar funciones en módulos

Una de las mayores ventajas de utilizar funciones es separar parte del código del programa principal. Si utilizamos nombres descriptivos para las funciones, podemos seguirlas más fácilmente y guardarlas en ficheros independientes que después podemos importar a nuestra área de trabajo. Este archivo con funciones es conocido como módulo y es la forma principal de reutilizar y compartir código entre desarrolladores.

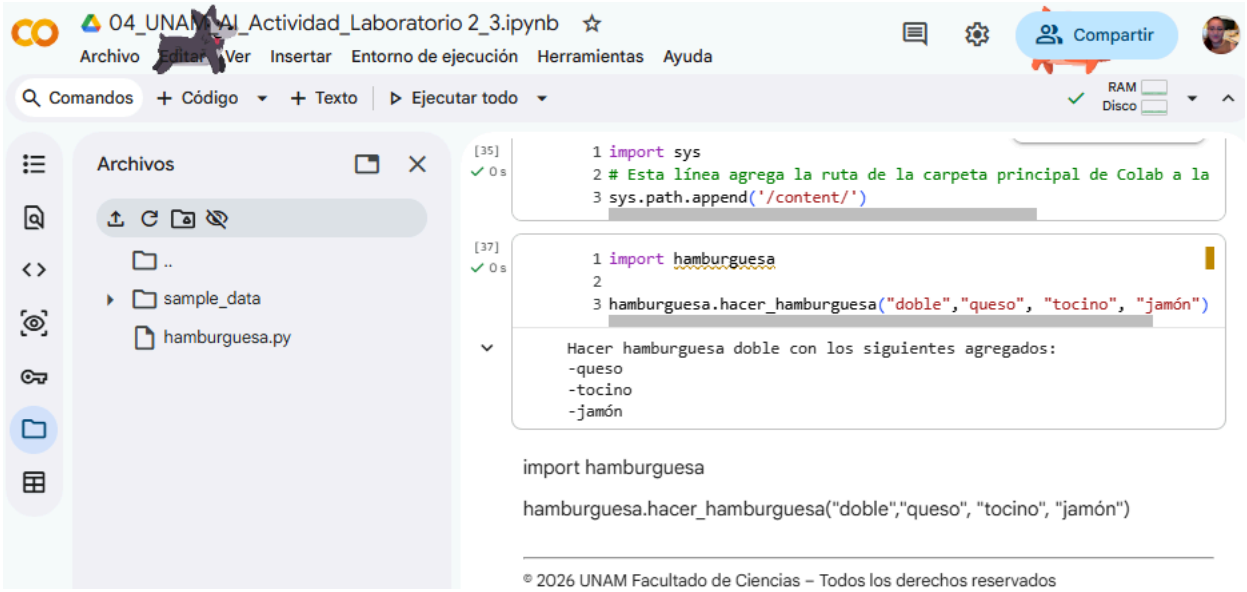
Cuando los módulos contienen muchas funciones específicas y fueron desarrolladas por otro programador externo a nuestro proyecto le llamamos librerías. Python, como lenguaje de programación, tiene cientos de librerías a su disposición, creadas por una comunidad activa que las mantiene actualizadas constantemente (Matthes, 2019).

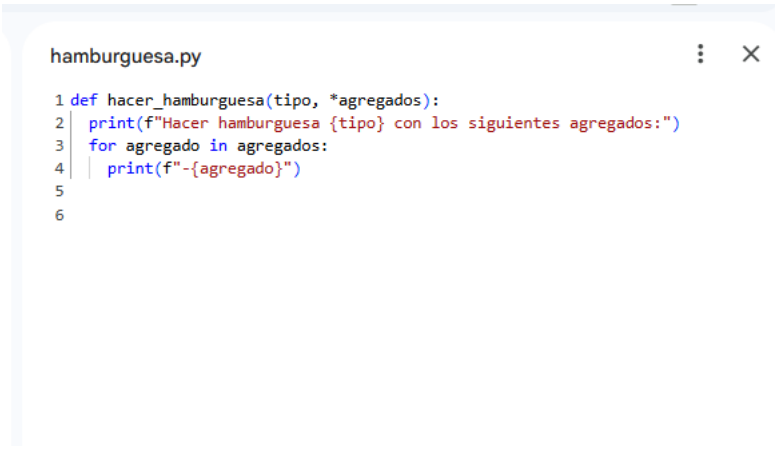
En un programa cuando queremos importar un módulo utilizamos la palabra reservada "import". Existen diferentes formas de realizar este proceso las cuales veremos continuación:

Importar un módulo entero

Para generar primeramente un módulo, necesitamos crear un documento de Python que solo contenga la función deseada (en este caso le llamaremos hamburguesa.py); una vez concluido este proceso, debemos copiar dicho archivo a la carpeta raíz donde se almacena nuestro proyecto. En el nuevo programa realizaremos la importación del módulo, quedando listo el entorno para hacer uso de del mismo. La sintaxis adecuada se muestra en el siguiente ejemplo.

En colab se hace agregando un archivo con la funcion y se sube al content, que es la carpeta que nos ofrece Google para uso de nuestro programa. Una vez que se escribe la funcion en ese archivo con la extensión .py se invoca con en el programa donde se usará la biblioteca sys para dar a conocer la ruta donde nuestro programa buscará la función.






```
import sys
# Esta línea agrega la ruta de la carpeta principal de Colab a la búsqueda de Python
sys.path.append('/content/')
```

```
import hamburguesa

hamburguesa.hacer_hamburguesa("doble","queso", "tocino", "jamón")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_6891/4132873553.py in <cell line: 0>()
----> 1 import hamburguesa
      2
      3 hamburguesa.hacer_hamburguesa("doble","queso", "tocino", "jamón")

ModuleNotFoundError: No module named 'hamburguesa'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

Próximos pasos: Explicar error

```
help(hacer_hamburguesa)
```

Importar una función específica

Para importar una función específica de un módulo se utiliza la siguiente sintaxis

```
from hamburguesa import hacer_hamburguesa

hacer_hamburguesa("doble","queso","salami","jamón")
```

También podemos importar varias de una sola vez

```
from hamburguesa import hacer_hamburguesa, hacer_licuado
```

```
help(hacer_licuado)
```

Asignar alias a las funciones

En ocasiones, es útil simplificar la nomenclatura de nuestros programas y más cuando utilizamos librerías de terceros. Para llevar a cabo esta tarea es muy útil asignarles alias a las funciones importadas, la forma de hacerlo se muestra en el siguiente ejemplo

```
from hamburguesa import hacer_hamburguesa as hh
```

Asignar alias a los módulos

De igual forma que se realizó con las funciones, es posible utilizar alias con los módulos. La forma adecuada se realizarlo

```
import hamburguesa as hm
```

Importar todas las funciones de un módulo

Podemos decirle a Python que importe todas las funciones de un módulo. Para realizar esta operación se utiliza en símbolo * y la sintaxis adecuada sería

```
from hamburguesa import *
```

Manejo de archivos usando la función open, close y write

De acuerdo con Lubanovic (2019), los denominados archivos planos son el ejemplo de archivo más simple y antiguo. Son solo una secuencia de bytes almacenados bajo un nombre y se pueden leer (read) o escribir (write)desde y hacia su propia localidad de memoria. Python facilita esas tareas, ya que su manejo de archivos se basa en las mismas operaciones populares de Unix.

Antes de leer o escribir un archivo, este debe abrirse:

archivo = open('nombre', 'modo')

A continuación, se ofrece una breve explicación para cada elemento:

archivo es el archivo devuelto por open()
'nombre' es el nombre en forma de cadena (string) del archivo. 'modo' también es una cadena que indica el tipo de archivo y lo que se desea hacer con él. La primera letra de 'modo' indica la operación:

'r' significa leer. 'w' significa escribir. Si el archivo no existe, se crea. Si el archivo existe, se sobrescribe. 'x' significa escribir, pero solo si el archivo aún no existe. 'a' significa agregar (escribir después del final) si el archivo existe. La segunda letra de modo es el tipo de archivo:

't' (o nada) significa texto. 'b' significa binario. Después de abrirse el archivo, se llama a las funciones correspondientes para leer o escribir datos, como más adelante se mostrará en algunos ejemplos. Por último, el archivo debe cerrarse.

A continuación, se realizará el ejemplo de un programa en el que se creará un archivo a partir de una cadena de Python. Las siguientes líneas de código permiten almacenar un poema en una cadena y calcular su longitud:

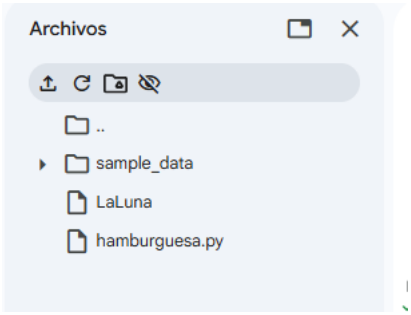
```
poema = '''<< LA LUNA
La luna se puede tomar a cucharadas
o como una cápsula cada dos horas.
Es buena como hipnótico y sedante
y también alivia
```

```
a los que se han intoxicado de filosofía.>> Jaime Sabines'''
len(poema)
```

```
190
```

Existen dos maneras distintas de escribir el poema anterior en un archivo; la primera es por medio de una sola llamada a la función write( ), la cual devuelve el número de bytes escritos sin agregar espacios ni saltos para una nueva línea, como sucederá en el segundo caso. A continuación, se escribe el poema completo bajo el nombre “LaLuna” utilizando write( ):

```
fout = open('LaLuna','wt')
fout.write(poema)
fout.close()
```



La segunda manera de escribir el poema en un archivo con el mismo nombre sería mediante la función print( ):

```
fout = open('LaLuna', 'wt')
print(poema)            # Esto lo imprime en la pantalla de Colab
print(poema, file=fout) # Esto lo guarda en el archivo
fout.close()
```

```
<< LA LUNA
La luna se puede tomar a cucharadas
o como una cápsula cada dos horas.
Es buena como hipnótico y sedante
y también alivia
a los que se han intoxicado de filosofía.>> Jaime Sabines
```

Lo anterior llevaría preguntarse ¿qué función debería usarse, write( ) o print( )? La diferencia entre ambos es que la segunda opción, print( ), agrega de forma predeterminada un espacio después de cada argumento y una nueva línea al final.

Lubanovic (2019) también explica que hay algunos casos en los que la cadena fuente es muy larga, por lo que es recomendable escribirla por fragmentos (chunks):

```
fout = open('LaLuna', 'wt')
size = len(poema)
offset = 0
chunk = 100
while True:
    if offset > size:
        break
    fout.write(poema[offset:offset+chunk])
    offset += chunk
fout.close()
```

El programa anterior escribe 100 caracteres en el primer ciclo y los últimos 83 en el siguiente. Por otra parte, si el archivo de “LaLuna” es valioso para ti, puedes usar el modo ‘x’ para protegerlo y evitar sobrescribir en él:

```
fout = open('LaLuna','xt')
```

```
---------------------------------------------------------------------------
FileExistsError                           Traceback (most recent call last)
/tmp/ipykernel_6891/2385602095.py in <cell line: 0>()
----> 1 fout = open('LaLuna','xt')

FileExistsError: [Errno 17] File exists: 'LaLuna'
```

Próximos pasos: Explicar error

Incluso puedes usar un manejador de excepciones:

```
try:
    fout = open('LaLuna', 'xt')
    fout.write('intentando escribir esto en el archivo')
except FileExistsError:
    print('¡El archivo LaLuna ya existe!')
```

Python también permite leer un archivo empleando las funciones read(), readline() o readlines().

Si se llama a read() sin especificar ningún argumento se extrae todo lo que hay en el archivo en una sola ejecución, como se muestra en el siguiente ejemplo:

```
finput = open('LaLuna', 'rt')
poema = finput.read()
finput.close()
len(poema)
```

```
190
```

Solo es importante tener cuidado al hacer lo mismo con archivos grandes, ya que un archivo de gigabytes consumiría al menos un gigabyte de memoria.

También es posible obtener un máximo de caracteres cada vez que se llama a read( ). A continuación, primero se reinicia a la cadena poema como vacía ('') y después se leen fragmentos (chunks) de 100 caracteres a la vez y se van agregando a poema para reconstruirlo:

```
poema = ''
finput = open('LaLuna', 'rt')
chunk = 100
while True:
    fragment = finput.read(chunk)
    if not fragment:
        break
    poema += fragment
finput.close()
len(poema)
```

```
190
```

Después de haber leído hasta el final, las siguientes llamadas a read( ) devolverán una cadena vacía (''), lo cual no sería un fragmento y se interpretaría como False en la condicional, rompiendo el bucle while True.

También se puede leer el archivo una línea a la vez utilizando la función readline( ). En el siguiente ejemplo, se añade a cada línea de la cadena poema para reconstruir el original:

```
poema = ''
finput = open('LaLuna', 'rt')
while True:
    line = finput.readline()
    if not line:
        break
    poema += line
finput.close()
len(poema)
```

```
190
```

Para un archivo de texto, incluso una línea en blanco tiene una longitud de uno (el salto para una nueva línea) y da como resultado True en la evaluación. Cuando se ha leído el archivo, readline( ) al igual que read() devuelve una cadena vacía, por lo que se considera False.

La forma más sencilla de leer un archivo de texto es mediante un iterador, devolviendo una línea a la vez. Es similar al ejemplo anterior, pero con menos código:

```
poema = ''
finput = open('LaLuna', 'rt')
for line in finput:
```

```
    poema += line
finput.close()
len(poema)
```

```
190
```

Todos los ejemplos anteriores construyeron eventualmente a poema en una única cadena. Por otro lado, la función readlines( ) lee una línea a la vez y devuelve una lista de cadenas correspondientes a cada línea del texto:

```
finput = open('LaLuna', 'rt')
lines = finput.readlines()
finput.close()
print(len(lines), 'líneas leídas')
for line in lines:
    print(line, end='')
```

```
6 líneas leídas
<< LA LUNA
La luna se puede tomar a cucharadas
o como una cápsula cada dos horas.
Es buena como hipnótico y sedante
y también alivia
a los que se han intoxicado de filosofía.>> Jaime Sabines
```

Con (line, end='')se le solicita a print( ) suprimir los saltos de línea automáticos porque ya están contemplados en el texto original. La última línea no lo hace, por lo que si se escribe algo más aparecerá directamente después de la última línea.

Si olvidas cerrar un archivo que has abierto, Python lo cerrará cuando ya no se haga referencia a él. Esto quiere decir que, si abres un archivo dentro de una función y no lo cierras explícitamente, este se cerrará automáticamente cuando finalice la función. Sin embargo, puede que hayas abierto el archivo en una función de larga duración o en la sección principal del programa, por lo que el archivo debe cerrarse para forzar que se completen las escrituras restantes.

Python tiene administradores de contexto para limpiar cosas como archivos abiertos. Para ello, se utiliza la sentencia with expresión as variable como se muestra a continuación:

```
with open('LaLuna', 'wt') as fout:
  fout.write(poema)
```

De esta manera, después de que el bloque de código bajo el administrador de contexto (en este caso, una línea) se completa (normalmente o por una excepción generada), el archivo se cierra automáticamente.

Cuando se trabaja con archivos de texto simples, el único nivel de organización es indicado son sus líneas. Pero en ocasiones, se requiere reestructurarlos de una manera más específica. También puede que desees guardarlos para que tu programa los utilice más tarde o incluso enviarlos a otro distinto.

Lubanovic (2019) señala que actualmente existen muchos formatos, pero dos de los principales son los siguientes:

- CSV (Comma-Separated Values): funcionan con un carácter separador o delimitador, como tabulación ('\ t'), coma (',') o barra vertical ('|').
- JSON (JavaScript Object Notation): este tipo de archivos separa los datos por puntuación. También existen otros formatos tales como XML y HTML, utilizados para aplicaciones web y YAML, creados con el propósito de que los datos sean más legibles para los humanos.

Los archivos delimitados (CSV) se utilizan a menudo como formato de intercambio para hojas de cálculo y bases de datos. Se pueden leer manualmente, una línea a la vez, dividiendo cada línea en campos con separadores de coma y agregando los resultados a estructuras de datos como listas y diccionarios. Sin embargo, es mejor usar el módulo csv estándar de Python, ya que analizar estos archivos puede volverse más complicado de lo que parece por las siguientes razones:

- Algunos tienen delimitadores alternativos además de una coma como '|' y '\ t' (tabulador).
- Otros tienen secuencias de escape, es decir, si el carácter delimitador puede aparecer dentro de un campo, todo el campo podría estar rodeado por comillas o precedido por algún carácter de escape.
- Los archivos tienen diferentes caracteres de final de línea; Unix usa '\ n' , Microsoft usa '\ r \ n' y Apple solía usar '\ r' pero ahora usa '\ n' .

Como prueba de manejo de archivos CSV, se escribe una lista de filas, cada una con una lista de columnas:

```
import csv
villanos = [
    ['Doctor', 'Doom'],
    ['Norman', 'Osborn'],
    ['Max', 'Eisenhardt'],
    ['Wilson', 'Fisk'],
    ['Sabah', 'Nur'],
     ]
with open('villanos', 'wt') as fout: #un administrador de contexto
     csvout = csv.writer(fout)
     csvout.writerows(villanos)
```

Esto crea el archivo de villanos con estas líneas:

Doctor, Doom

Norman, Osborn

Max, Eisenhardt

Wilson, Fisk

Sabah, Nur

Ahora, se procede a leer dicho archivo:

```
import csv
with open('villanos', 'rt') as finput:  #administrador de contexto
    cinput = csv.reader(finput)
    villanos = [fila for fila in cinput]  # Se usa una lista de comprensión
    print(villanos)
```

```
[['Doctor', 'Doom'], ['Norman', 'Osborn'], ['Max', 'Eisenhardt'], ['Wilson', 'Fisk'], ['Sabah', 'Nur']]
```

En este caso, se aprovecha la estructura creada por la función reader( ), la cual crea filas en el objeto sin que pueden extraerse con un bucle for. Usando reader( ) y writer () con sus opciones predeterminadas, las columnas están separadas por comas y las filas por líneas de avance.

De acuerdo con Lubanovic (2019), la notación de objetos JavaScript (JSON) se ha convertido en un formato de intercambio de datos muy popular, más allá de sus orígenes en JavaScript. El formato JSON es un subconjunto de JavaScript y, a menudo, también la sintaxis legal de Python. Su estrecha relación con Python lo convierte en una buena opción para el intercambio de datos entre programas.

En Python hay un módulo JSON principal, bajo su mismo nombre, el cual puede codificar (dumps) los datos en una cadena JSON o decodificar (cargar) una cadena JSON de nuevo a datos. En este ejemplo, se crea una estructura de datos de Python que contiene los siguientes datos:

```
menu = \
{
    "desayuno": {
        "horario": "7-11",
        "alimentos": {
             "huevos revueltos": "$40.00",
             "hotcakes": "$30.00"
         }
     },
    "almuerzo" : {
        "horario": "11-3",
        "alimentos": {
            "hamburguesa": "$60.00"
         }
     },
    "cena": {
        "horario": "3-10",
        "alimentos": {
            "spaghetti": "$50.00"
         }
     }
}
```

Ahora, se codifica la estructura de datos menu en una cadena JSON menu_json utilizando dumps( ):

```
import json
menu_json = json.dumps(menu)
menu_json
```

```
'{"desayuno": {"horario": "7-11", "alimentos": {"huevos revueltos": "$40.00", "hotcakes": "$30.00"}}, "almuerzo": {"horario":
"11-3", "alimentos": {"hamburguesa": "$60.00"}}, "cena": {"horario": "3-10", "alimentos": {"spaghetti": "$50.00"}}}'
```

Y como última prueba para este tipo de formato, se convierte la cadena JSON menu_json de nuevo en una estructura de datos de Python menu2 usando loads():

```
menu2 = json.loads(menu_json)
menu2
```

```
{'desayuno': {'horario': '7-11',
  'alimentos': {'huevos revueltos': '$40.00', 'hotcakes': '$30.00'}},
 'almuerzo': {'horario': '11-3', 'alimentos': {'hamburguesa': '$60.00'}},
 'cena': {'horario': '3-10', 'alimentos': {'spaghetti': '$50.00'}}}
```

menu y menu2 son diccionarios con las mismas claves y valores, y como ocurre con los diccionarios estándar, el orden en el que se obtienen las claves varía.

Objetos y clases

De acuerdo con Matthes (2019), en la programación orientada a objetos, a la creación de un objeto a partir de una clase se le denomina instanciación y es así como se trabaja con instancias de una clase. En los ejemplos posteriores se escribirán clases y se crearán instancias para ellas. El programador puede especificar el tipo de información que se puede almacenar en cada caso y definir las acciones que se pueden tomar para cada uno de ellos.

Las clases también facilitan la programación a medida que se asumen desafíos cada vez más complejos. Cuando se escribe código basado en el mismo tipo de lógica, el trabajo se vuelve comprensible para el autor y otros programadores que deseen usarlo. Esto a su vez, da lugar a la colaboración permitiendo que el proyecto logre extenderse y tenga mayor alcance.

Casi cualquier cosa puede modelarse usando clases. Se comenzará escribiendo una clase simple, Perro, que representa a un perro, no a un perro en particular, sino a cualquier perro. Ahora, es necesario hacerse la siguiente pregunta: ¿qué es lo que se sabe sobre la mayoría de los perros domésticos? Una respuesta sería que todos tienen nombre y edad. También se sabe que la mayoría de los perros pueden sentarse y darse la vuelta.

Esas dos piezas de información (nombre y edad) y esos dos comportamientos (sentarse y darse la vuelta) se incluirán en la clase Perro porque son comunes en la mayoría de los perros. Esta clase indicará a Python cómo crear un objeto que represente a un perro.

En este primer ejemplo, cada instancia creada a partir de la clase Perro almacenará un nombre y una edad, y se le dará a cada perro la capacidad de sentarse() y darse_la_vuelta():

```
class Perro:
  """Un intento simple de modelar un perro."""
  def __init__(self, nombre, edad):
    """Inicializar los atributos de nombre y edad."""
    self.nombre = nombre
    self.edad = edad
  def sentarse(self):
    """Simula un perro sentado como respuesta a una orden."""
    print(f"{self.nombre} ahora está sentado.")
  def darse_la_vuelta(self):
    """Simular darse la vuelta como respuesta a un comando."""
    print(f"{self.nombre} se dio la vuelta!")
```

En la línea 1 se define una clase llamada Perro. Por convención, los nombres en mayúscula se refieren a clases en Python. No hay paréntesis en la definición de la clase porque estamos creando esta clase desde cero. En la línea 2 se escribe una cadena de documentación que describe lo que hace esta clase.

Una función que forma parte de una clase es un método, el cual posee las mismas propiedades que cualquier función. El método __ init _() *en la línea 3 es un método especial que Python ejecuta automáticamente cada vez que se crea una nueva instancia basada en la clase Perro. Este método tiene dos guiones bajos ()* al inicio y al final, esto ayuda a evitar que los nombres de métodos predeterminados de Python entren en conflicto con los nombres de sus métodos.

Se define el método el __ init __() para tener tres parámetros: self, nombre y edad. El parámetro self es obligatorio en la definición del método y debe ir primero antes que los demás parámetros. Debe incluirse porque cuando Python llama a este método (para crear una instancia de Perro), la llamada al método pasará automáticamente el argumento self. Cada llamada al método asociada con una instancia

pasa automáticamente al argumento self, lo cual es una referencia a la instancia en sí; dándole acceso a los atributos y métodos de la clase.

Cuando se crea una instancia de Perro, Python llama al método __ init __() de la clase Perro. Entonces se pasa a Perro() un nombre y una edad como argumentos; self se pasa automáticamente, por lo que no es necesario hacerlo manualmente. Entonces, cada vez que desee hacerse una instancia de la clase Perro, solo se proporcionarán los valores para los dos últimos parámetros, nombre y edad.

Las dos variables definidas en la línea 4 tienen cada una el prefijo self, por lo que estarán disponibles para cada método en la clase e igualmente será posible acceder a ellas a través de cualquier instancia creada a partir de la clase. La línea self.name = name toma el valor asociado con el nombre del parámetro y lo asigna al nombre de la variable, que luego se adjunta a la instancia que se está creando. El mismo proceso ocurre con self.age = age. Las variables a las que se puede acceder a través de instancias como esta se denominan atributos.

La clase Perro tiene otros dos métodos definidos a partir de la línea 6: sentarse( ) y darse_la_vuelta(). Debido a que estos métodos no necesitan información adicional para ejecutarse, simplemente se definen por un único parámetro, self. Las instancias que se realicen posteriormente tendrán acceso a estos métodos. En otras palabras, serán capaces de sentarse y darse la vuelta. Por ahora, sentarse() y darse_la_vuelta() solo imprimen un mensaje que indica que el perro está sentado o que se ha dado la vuelta. Pero el concepto puede extenderse a situaciones realistas. Por ejemplo, si esta clase fuera parte de un juego de computadora real, estos métodos contendrían código para hacer que un perro animado se siente y se dé la vuelta. O si esta clase se codificó para controlar un robot, estos métodos dirigirían los movimientos que hacen que un perro robótico se siente y se dé la vuelta.

La clase Perro es un conjunto de instrucciones que le dice a Python cómo crear instancias individuales que representen perros específicos. Ahora, se procederá a crear una instancia que represente a un perro en específico:

```
mi_perro = Perro ('Chad', 2)
print(f"El nombre de mi perro es {mi_perro.nombre}.")
print(f"Mi perro tiene {mi_perro.edad} años")
```

```
El nombre de mi perro es Chad.
Mi perro tiene 2 años
```

La clase Perro que usamos aquí es la que acabamos de escribir en el ejemplo anterior. En la línea 1 se le indica a Python que cree un perro cuyo nombre sea 'Chad' y cuya edad sea 2. Cuando Python lee esta línea, llama al método __ init __() en Perro con los argumentos 'Chad' y 2.

El método __ init __() crea una instancia que representa a este perro en particular y establece los atributos de nombre y edad usando los valores que proporcionamos. Python luego devuelve una instancia que representa a este perro. Entonces se asigna esa instancia a la variable mi_perro.

Para acceder a los atributos de una instancia, se usa la notación de puntos (.). En la línea 2 se accede al valor del atributo nombre de mi_perro escribiendo mi_perro.nombre. Este es el mismo atributo al que se hace referencia como self.name en la clase Perro. En la línea 3 se utiliza el mismo enfoque para trabajar con el atributo edad.

Después de crear una instancia de la clase Perro, se puede usar la notación de puntos para llamar a cualquier método definido en la clase. Con el siguiente código el perro se sienta y da la vuelta:

```
mi_perro.sentarse()
mi_perro.darse_la_vuelta()
```

```
Chad ahora está sentado.
Chad se dio la vuelta!
```

Para llamar a un método, se proporciona el nombre de la instancia (en este caso, mi_perro) y el método que desea llamar, separados por un punto. Cuando Python lee mi_perro.sentarse(), busca el método sentarse( ) en la clase Perro y ejecuta ese código. La línea de mi_perro.darse_la_vuelta() se interpreta de la misma manera.

Esta sintaxis es bastante útil, ya que una vez que se les han asignado nombres descriptivos apropiados a los atributos y métodos, tales como nombre, edad, sentarse( ) y darse_la_vuelta(), se puede inferir fácilmente lo que se supone que debe hacer cada bloque de código, incluso sin haberlo visto antes.

Desafío

Escribe un programa en Python que realice lo siguiente:

- Codifica una función que reciba una lista como argumento y que genere un diccionario como salida.

- El diccionario generado debe tener las siguientes llaves (claves): nombre, apellido y edad.
- Genera la lista de entrada con valores ficticios de posibles personas reales.
- Ejecuta el programa tres veces con valores diferentes y almacena todos los resultados en la estructura de datos más adecuada para la situación planteada.
- Escribe el diccionario del ejercicio anterior en un archivo CSV o JSON.
- Usando alguna de las funciones de lectura de Python, despliega el contenido de lo anterior.
- Por medio de la clase persona, modela la información del ejercicio anterior y crea una instancia que contenga tu nombre, apellido y edad como atributos.

Problema

```
import csv
import json

#Clase Persona con atributos nombre, apellido y edad
class Persona:
  def __init__(self, nombre, apellido,edad) -> None:
     self.nombre = nombre
     self.apellido =apellido
     self.edad =edad
  def __str__(self):
     return f"Objeto Persona:{self.nombre}{self.apellido}, Edad: {self.edad}"

# Función para generar un diccionario a partir de una lista de nombre, apellido y edad
# EJEMPLO DE UNA POSIBLE FUNCION
def generar_diccionario_personas(lista_personas):
    diccionario_personas = []
    for persona in lista_personas:
        nombre, apellido, edad = persona
        diccionario_personas.append({"nombre": nombre, "apellido": apellido, "edad": edad})
    return diccionario_personas

# Lista de personas ficticias
lista_personas1 = [("Juan", "Pérez", 30), ("María", "Gómez", 25), ("Luis", "Martínez", 35)]
lista_personas2 =[("alumno 1", "Pérez", 30), ("María", "Gómez", 25), ("Luis", "Martínez", 35)]
lista_personas3 =[("alumno 2", "Pérez", 30), ("María", "Gómez", 25), ("Luis", "Martínez", 35)]

# Generar diccionarios
diccionario_personas1 = generar_diccionario_personas(lista_personas1)
diccionario_personas2 = generar_diccionario_personas(lista_personas2)
diccionario_personas3 = generar_diccionario_personas(lista_personas3)

# Almacenar los resultados en una lista de diccionarios
resultados =[]
resultados.extend(diccionario_personas1)
resultados.extend(diccionario_personas2)
resultados.extend(diccionario_personas3)


# Escribir en un archivo CSV
with open ('resultados.csv','w',newline='', encoding='utf-8') as csvfile:
  campos =['nombre','apellido','edad']
  writer = csv.DictWriter(csvfile,fieldnames=campos)
  writer.writeheader()
  writer.writerows(resultados)

# Escribir en un archivo JSON
with open('resultados.json','w') as jsonfile:
  json.dump(resultados,jsonfile)


# Leer el contenido del archivo CSV
print("contenido del archivo CSV:")
with open('resultados.csv','r') as csvfile:
  reader = csv.reader(csvfile)
  for row in reader:
    print(row)


# Leer el contenido del archivo JSON
print("\nContenido del archivo JSON:")
```

```
with open('resultados.json','r') as jsonfile:
  data = json.load(jsonfile)
  print(data)


# Definir la clase Persona
class Persona:
  def __init__(self, nombre, apellido, edad):
    self.nombre = nombre
    self.apellido = apellido
    self.edad = edad


# Crear una instancia de la clase Persona
mi_persona = Persona("Juan", "González", 25)
print("\nInformación de la instancia de Persona:")
print("Nombre:", mi_persona.nombre)
print("Apellido:", mi_persona.apellido)
print("Edad:", mi_persona.edad)
```

```
contenido del archivo CSV:
['nombre', 'apellido', 'edad']
['Juan', 'Pérez', '30']
['María', 'Gómez', '25']
['Luis', 'Martínez', '35']
['alumno 1', 'Pérez', '30']
['María', 'Gómez', '25']
['Luis', 'Martínez', '35']
['alumno 2', 'Pérez', '30']
['María', 'Gómez', '25']
['Luis', 'Martínez', '35']

Contenido del archivo JSON:
[{'nombre': 'Juan', 'apellido': 'Pérez', 'edad': 30}, {'nombre': 'María', 'apellido': 'Gómez', 'edad': 25}, {'nombre': 'Luis', '

Información de la instancia de Persona:
Nombre: Juan
Apellido: González
Edad: 25
```

Conclusiones del estudiante

Haz doble clic (o ingresa) para editar

© 2026 UNAM Facultad de Ciencias – Todos los derechos reservados